A Decentralized Approach Model (ADAM) for Solving PoW Puzzles of Blockchains to Decentralize Network Hash Power and Reward Distribution

Bedri Özgür Güler
bedriguler@gmail.com

> **NOTE**
>
> *This paper presents the conceptual framework and actual production implementation of the ADAM (A Decentralized Approach Model) consensus mechanism. Originally drafted for Ravencoin's algorithm discussions, the model has been fully implemented and adapted for the KristaTech blockchain.*

Abstract

In this paper, we analyze the source of hash power centralization in Proof-of-Work (PoW) consensus blockchains, where Application-Specific Integrated Circuits (ASICs) and Field-Programmable Gate Arrays (FPGAs) dominate block production. We propose a decentralized approach model (ADAM) that replaces the traditional "hash power race" with a cooperative, distributed problem-solving model. By electing a random pool of miners and a coordinator for each block slot, distributing lightweight parallelized PoW puzzles, and using a sequential hashing chain to generate the block hash, ADAM ensures that building ASICs is economically infeasible. Additionally, we integrate this model with Proof-of-Stake (PoS) to create a secure, dual-signature hybrid consensus topology.

1. Introduction

Satoshi Nakamoto's original Bitcoin protocol [1] relies on a competitive race where multiple parties solve a double-SHA256 PoW puzzle. The first party to publish a valid solution receives the block reward. This competitive dynamic inherently rewards scale: the more hash power a miner controls, the higher their probability of winning the block reward.

This race quickly transitioned from CPUs to GPUs, then to FPGAs, and finally to ASICs. Various projects attempt to design "ASIC-resistant" algorithms by increasing memory requirements or dynamically changing the hashing sequence (e.g., X16R) [2]. However, once a cryptocurrency reaches a high enough valuation, developing custom ASICs becomes highly profitable, leading to the inevitable centralization of network hash power.

Hash power centralization poses severe risks:

1. **51% Attacks**: Single entities or cartels controlling dominant pools can rewrite transaction history.
2. **Economic Exclusion**: Retail miners using commodity hardware (CPUs/GPUs) are priced out, reducing network distribution.

3. **ECOLOGICAL WASTE**: Massive amounts of energy are wasted by competing miners whose solutions are discarded because they did not win the race.

ADAM (A Decentralized Approach Model) solves these problems by replacing the “competitive race” with a **cooperative distribution of hash power and rewards** among randomly selected nodes.

---

2. A Decentralized Approach Model (ADAM)

Motivation

The primary motivation of ADAM is to democratize mining by dividing a complex consensus puzzle into simpler, parallelized partial problems, distributing them to a group of nodes elected by network consensus, and combining their proofs into a single cooperative block validation.

ADAM solves three critical problems:

1. **ASIC Domination**: Because the subset of eligible miners changes randomly for each block, and the required hashing algorithms are dynamically assigned, static ASIC designs cannot achieve a dominant competitive advantage.
2. **Energy Efficiency**: Instead of the entire network racing and wasting electricity, only the elected miners perform work for a given block slot.
3. **Wealth Distribution**: Block rewards are shared among all participants in the cooperative round (the elected miners and the coordinator), achieving an “equal pay for equal work” distribution.

---

3. Mathematical Formulation

To bridge the theoretical concept of cooperative problem solving with the actual blockchain code, we present both the abstract operator formulation and its concrete production implementation.

3.1. Theoretical Framework (Cryptographic Function Composition)

We define a classical mathematical framework using cryptographic function composition to represent block header states and cooperative hashing operations.

1. Block Header Representation

Let $B(q, s)$ represent the serialized state of a block header, parameterized by the nonce q and the `extraNonce` s in the coinbase transaction (which is cryptographically committed to by the block’s Merkle root). For simplicity, we denote the block state as B, or B_k when customized for the k-th miner.

2. Hashing Operations

Applying a hashing function H (such as SHA-256 or X11) to a block header state is defined as:

$$H(B(q, s)) = \text{hash}(B(q, s)) \in \{0, 1\}^{256}$$

where H can represent a single hashing algorithm or a chained sequence of distinct algorithms.

3. The Theoretical Cooperative Problem

The cooperative validation model divides the target Proof-of-Work (PoW) puzzle into N parallel sub-problems, each solved by an elected miner $i \in \{1, \ldots, N\}$:

$$h_i = H_i(C_i(q_i))$$

where:

- N is the size of the elected miner pool.
- H_i is the specific hashing function permuted and assigned to miner i.
- $C_i(q_i)$ represents the customized puzzle challenge for miner i, incorporating the block's rolling seed, the miner's public key, and their private nonce q_i.
- h_i is the lightweight puzzle hash solution computed by miner i below their individual difficulty target.

4. Cryptographic Binding (Chaining Composition)

To bind the block header cryptographically to the work of all elected miners, we define a sequential, non-linear composition function $\mathscr{F}$ over the block header B:

$$H_{\text{block}} = \mathscr{F}(B) = \mathscr{H}_N \circ \mathscr{H}_{N-1} \circ \ldots \circ \mathscr{H}_1(B)$$

where each step $\mathscr{H}_i$ is a composition function parameterized by the algorithm permutations and coprime multipliers m_i derived from the previous state:

$$\mathscr{H}_i(X) = (H_i^{(1)}(H_i^{(2)}(H_i^{(3)}(X) \times (i+1)) \times (i+1)) \times m_i) \pmod{2^{256}}$$

This sequential chaining ensures that the block hash H_{block} is valid if and only if every single elected miner i has completed their corresponding lightweight PoW puzzle. Altering or omitting any contribution breaks the chain, rendering the final block hash invalid.

3.2. Implemented Production Mathematics (KristaTech Blockchain)

In a live blockchain database, floating-point representations and sum-of-product hashes are difficult to validate deterministically across distributed nodes. To solve this, the KristaTech production codebase implements the theoretical linear combination using two concrete structures: **Lightweight PoW Puzzles** and a **Stateless Hashing Chain**.

1. Lightweight Puzzle Solving

Each elected miner $i \in \{0, \ldots, N-1\}$ must prove they performed work by solving a lightweight puzzle:

$$\text{PuzzleHash}_i = \text{CalculateAdamPuzzleHash}(\text{algoIndex}_i, \text{Seed}_H \| \text{MinerPubKey}_i \| \text{Nonce}_i)$$

Subject to the target difficulty limit:

$$\text{PuzzleHash}_i \leq \text{scaledTarget}$$

where:

- Seed_H is the rolling VRF seed for the current block height.
- MinerPubKey_i is the public key of the elected miner.
- The starting difficulty target limit (`powLimit`) is set to `~UINT256_ZERO >> 20` on Mainnet and Testnet (equivalent to `0x1e0ffff0`). This prevents microsecond blocks and consensus splits during genesis bootstrapping.
- scaledTarget = min(Target $\lll$ activeShift, powLimit) (on Mainnet and Testnet; Regtest sets `scaledTarget = ~UINT256_ZERO`). This bit-shift relaxes the difficulty, ensuring the puzzle remains lightweight and solvable within the target block spacing (30 seconds):
  - activeShift=nAdamDifficultyShiftV1 (`10`) in Fallback Mode (`nVersion == 11`, `!NetworkUpgradeActive(nHeight, Consensus::UPGRADE_MODELD)`, or `!NetworkUpgradeActive(nHeight, Consensus::UPGRADE_ADAM_V2)`).
  - activeShift=nAdamDifficultyShiftV2 (`6`) in Standard Mode (`nVersion >= 12`) once both `Consensus::UPGRADE_MODELD` and `Consensus::UPGRADE_ADAM_V2` are active (effective at Block `2,200` on Mainnet and Block `500` on Testnet).

The hashing algorithm index (algoIndex_i) is dynamically assigned:

- **Fallback Mode (Version 11)**: $\text{algoIndex}_i = \text{Hash}(\text{Seed}_H \| \text{MinerPubKey}_i) \pmod{18}$, using one of 18 energy-efficient hash functions.
- **Standard Mode (Version 12)**: Uses a 3-permutation selector scheme GetAdam3PermutationAlgos(hashPrevBlock, MinerPubKey_i) that deterministically selects 3 distinct hashing algorithms (algo1, algo2, and algo3) out of 18 available algorithms based on the previous block's hash and the miner's public key. The solver compounds the three algorithms:

  $$\text{PuzzleHash}_i = \text{algo1}((\text{minerIdx} + 1) \times \text{algo2}((\text{minerIdx} + 1) \times \text{algo3}(\text{Challenge}))) \pmod{2^{256}}$$

2. Stateless Hashing Chain (Block Hashing)

To bind the block header cryptographically to the work of all elected miners, the block hash (H_{block}) is computed by sequentially chaining the hashing operations of all elected miners.

For a serialized block header S:

1. For each round $i \in \{0, \ldots, M-1\}$ (where M = vAdamMiners.size() − 1 in Version 11 Fallback Mode and M = vAdamMiners.size() in Version 12 Standard Mode), we calculate an odd coprime multiplier m_i to preserve hash entropy:

   $$\text{roundHash}_i = \text{Hash}(\text{hashPrevBlock} \| i)$$

   $$m_i = \max(\text{roundHash}_i[0] \mid 1, 3)$$

2. In Version 11 Fallback Mode, each round i applies a single hash $\text{CalculateAdamPuzzleHash}(\text{algo}_i, \cdot) \times m_i \pmod{2^{256}}$ with $\text{algo}_i = \text{Hash}(\text{hashPrevBlock} \| \text{vAdamMiners}[i]) \pmod{18}$. In Version 12 Standard Mode, the 3-permutation rounds (`GetAdam3PermutationAlgos(hashPrevBlock, vAdamMiners[i])`) are chained sequentially:
   - **Round 0**:
     - Derive algo1, algo2, and algo3 for miner 0.

- Compute:

  $H_0^{(3)}$ = CalculateAdamPuzzleHash(algo3, S)

  $H_0^{(2)}$ = CalculateAdamPuzzleHash(algo2, $(H_0^{(3)} \times 1) \pmod{2^{256}}$)

  H_0 = CalculateAdamPuzzleHash(algo1, $(H_0^{(2)} \times 1) \pmod{2^{256}}$)

- Apply multiplier:

  $H_{\text{prev}} = (H_0 \times m_0) \pmod{2^{256}}$

- **Round** $i > 0$:

  - Derive algo1, algo2, and algo3 for miner i.
  - Compute:

    $H_i^{(3)}$ = CalculateAdamPuzzleHash(algo3, H_{prev})

    $H_i^{(2)}$ = CalculateAdamPuzzleHash(algo2, $(H_i^{(3)} \times (i + 1)) \pmod{2^{256}}$)

    H_i = CalculateAdamPuzzleHash(algo1, $(H_i^{(2)} \times (i + 1)) \pmod{2^{256}}$)

  - Apply multiplier:

    $H_{\text{prev}} = (H_i \times m_i) \pmod{2^{256}}$

3. The final output is the block hash:

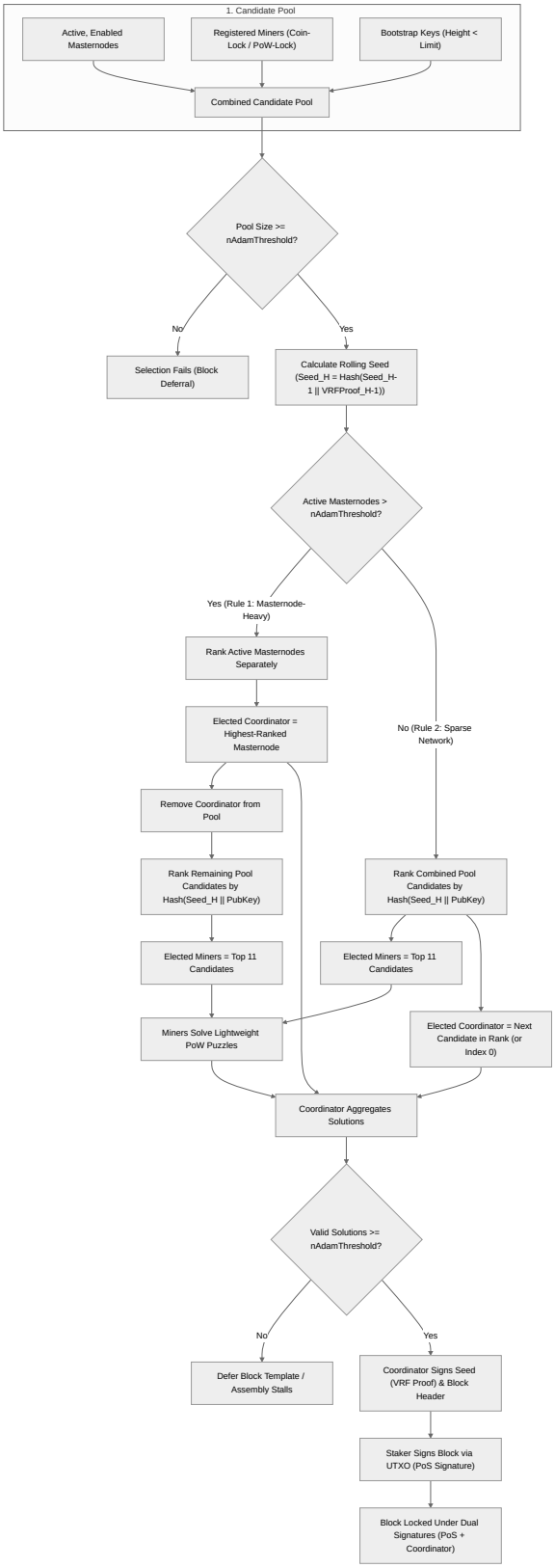
   $H_{\text{block}} = H_{\text{prev}}$

This sequential, non-linear hashing chain enforces that a block is only valid if it contains the correct mathematical signature of all cooperative mining rounds.

---

4. The Implemented Production Pipeline

The consensus flow is structured as a **Cooperative Hybrid Round** with the following pipeline:



1. Active Node Pool

The pool of active nodes (`GetAdamMinerPool()`) is derived deterministically on-chain from active Masternodes (with unspent 4,200 KRISTA collateral and on-chain heartbeat pings within the last 2,880 blocks) and active registered miners:

- **Dual-Path Miner Registration**: Nodes can register as miners using one of two methods:

- **Coin-Lock Registration**: Requires locking at least 250 KRISTA (`MINER_REGISTRATION_LOCK_AMOUNT = 250 * COIN`) in a CLTV registration output for a lock time of at least `nRegPeriod` blocks (valid at all heights via manual `registerminer "<addr>" "coin"` RPC and consensus validation, and automatically selected by background `AutoRegisterMiner` from block 2,200 onward).
- **PoW-Lock Registration**: Requires solving an out-of-band Proof-of-Work challenge mapped to a recent block hash, submitted in a CLTV registration output locking **10 KRISTA** (`POW_LOCK_REGISTRATION_AMOUNT = 10 * COIN`, funded by inputs with ≥ 100 confirmations) or **0 KRISTA** (`Zero-Coin PoW` when the active pool has < 13 keys) valid for at least `nRegPeriod` blocks.

- **Registration Period (`nRegPeriod`)**: The validity period for miner registration is `2880` blocks on Mainnet, and `2880` blocks on Testnet/Regtest initially (reduced to `100` blocks once the Model D network upgrade is active).
- **Genesis Bootstrapping (Bootstrap Miner Key Pool)**: To prevent chain stalls during the early phase when few masternodes or registered miners are active:
  - **Mainnet**: As long as the block height is below `nAdamBootstrapLimit` (44,850 blocks), the pool automatically registers the public keys of the block producers from blocks 1 to 199. Note that the ADAM consensus itself starts at block 200, followed by POMBL at block 2,000 and Model D at block 2,200.
  - **Testnet**: As long as the block height is below `nAdamBootstrapLimit` (100,000 blocks), the pool automatically registers the public keys of the block producers from blocks 1 to 199 (ADAM starts at block 200, POMBL at block 400, and Model D at block 500).
- **Regtest**: The pool automatically includes 15 deterministic bootstrap public keys to facilitate automated testing.

2. Deterministic Leader Election (SSLE) & Dual-Track Quotas

For each block height H where the ADAM network upgrade (`Consensus::UPGRADE_ADAM`) is active, the network uses a deterministic single secret leader election (SSLE) algorithm (`SelectAdamNodes`).

- The roll uses a rolling seed:

  $$\text{Seed}_H = \text{Hash}(\text{Seed}_{H-1} \| \text{VRFProof}_{H-1})$$

- Each node in the pool is ranked:

  $$\text{Rank}_i = \text{Hash}(\text{Seed}_H \| \text{PubKey}_i)$$

- Let T_{active} be the total count of active Masternodes on the network, and $T_{\text{threshold}}$ be the active quorum threshold (`GetAdamThreshold(nHeight)`, which is `5` below `nAdamThresholdBootstrapLimit` [50,000 on Mainnet, 100,000 on Testnet, 1,000 on Regtest] and `nAdamThreshold` [`7` on Mainnet/Regtest, `3` on Testnet] thereafter):
  - **Rule 1 (Masternode-Heavy: $T_{\text{active}} > T_{\text{threshold}}$)**:
    - Active masternodes are ranked separately: $\text{Rank}_{\text{mn},i} = \text{Hash}(\text{Seed}_H \| \text{PubKey}_{\text{mn},i})$.
    - The highest-ranked masternode is elected as the **Coordinator**.
    - The Coordinator is excluded from the validator candidate pool.
    - When `nHeight >= consensus.nAdamDualTrackHeight` (`UPGRADE_ADAM` = block 200 on Mainnet/Testnet, block 1 on Regtest), the M = nAdamMinersCount = 11 **Miners** are elected via `ElectDualTrackValidators`: **Track 1 (8 seats)** allocates **7 seats** to Active Masternodes (`4,200`

`KRISTA`) and **1 seat** to Coin-Lock miners (`250 KRISTA`) with immediate intra-Track-1 spillover, while **Track 2 (3 seats)** is allocated to Open PoW miners (`10 KRISTA` PoW-Lock / `0 KRISTA` Zero-Coin PoW / Bootstrap) with bidirectional inter-track spillover.

- **Rule 2 (Sparse Network: $T_{\text{active}} \leq T_{\text{threshold}}$)**:
  - When `nHeight >= consensus.nAdamDualTrackHeight`, the top $M = 11$ **Miners** are elected via `ElectDualTrackValidators` (`7 Active MN + 1 Coin-Lock + 3 Open PoW` with spillover), and the highest-ranked remaining candidate is elected as the **Coordinator**.

- **Activation Modes**:
  - **Fallback Mode (Block Version 11)**: Active when the `Consensus::UPGRADE_ADAM` network upgrade is active (height 200 on Mainnet/Testnet/Regtest) and the `Consensus::UPGRADE_POMBL` upgrade is inactive. It expects a flexible list of miners (between `nAdamThreshold + 1` and `14` total nodes), with the last miner serving as the Coordinator.
  - **Standard Mode (Block Version 12)**: Active deterministically when the `Consensus::UPGRADE_POMBL` network upgrade is active (height 2000 on Mainnet, 400 on Testnet, 300 on Regtest). It enforces exactly `nAdamMinersCount = 11` miners and 1 distinct Coordinator.

3. Solving Phase (Lightweight PoW)

Elected miners solve the lightweight PoW puzzle and broadcast their partial solution over the P2P network. The solution contains:

- `nNonce`: The 32-bit nonce that solves the puzzle.
- `vchSig`: The miner's cryptographic signature on the solved `puzzleHash`, verified using the `VerifyBLSWithECDSAFallback` scheme.

4. Orphan Solution Cache

To prevent block assembly stalls caused by network propagation latency, nodes cache solutions received out of order (`mapOrphanAdamSolutions`). When a node receives a puzzle solution for a block height whose predecessor is not yet processed, it holds it in the orphan cache and processes it once the predecessor block is added to the block index.

5. Aggregation, Verification Quorum, Standby Handover & Time-Relaxed Threshold

The Coordinator aggregates the solutions. To prevent sabotage or offline node issues, the network enforces a quorum threshold (T = GetAdamThreshold($nHeight$)):

- **Bootstrap Phase (`nHeight < nAdamThresholdBootstrapLimit`, i.e., 50,000 on Mainnet, 100,000 on Testnet, 1,000 on Regtest)**: Requires at least `5` valid solutions (`3` on Testnet after bootstrap) from the 11 elected miners.
- **Post-Bootstrap (`nHeight >= nAdamThresholdBootstrapLimit`)**: Requires at least `nAdamThreshold = 7` valid solutions on Mainnet/Regtest (`3` on Testnet).
- **Standby Validator Seat Handover ($t \geq 60$s)**: If primary committee seats remain silent for `nTimeElapsed >= 60s` (`ADAM_COMMITTEE_FALLBACK_TIMEOUT_SECS = 60`, advancing in `ADAM_COMMITTEE_FALLBACK_SLOT_SECS = 30` slots), `ResolveAdamCommitteeForTime` /

`ValidateAdamCommitteeForTime` allow the active Coordinator to replace unsolved primary seats strictly with same-track standby candidates (`MN -> MN`, `Coin-Lock -> Coin-Lock`, `Open PoW -> Open PoW`) in sequential rank order verified against the block's `CCoinsViewCache* pView`.

- **Time-Relaxed Quorum Threshold ($t \geq 120$s)**: If the committee still cannot reach the baseline threshold T, `GetAdamEffectiveThreshold(nHeight, nTimeElapsed, nActiveMasternodes)` relaxes the threshold by `1` every `60s` starting at $t = 120$s down to `ADAM_MIN_RELAXED_THRESHOLD = 5` (`> 4` non-MN seats, ensuring at least 1 Active Masternode is always required when post-bootstrap OR active MNs ≥ 7, or down to `ADAM_MIN_RELAXED_THRESHOLD_BOOTSTRAP = 3` during sparse bootstrap when `nHeight < nAdamThresholdBootstrapLimit` and active MNs < 7).
- **Equal Work, Equal Reward & 50% Automatic Protocol Burn**: Under Model D, every solved seat (`!vAdamSolutions[i].empty()`, primary or same-track standby) receives 100% of its own validator track's per-seat share of the 40% Participant Validator pool (20% across the 7 Masternode Validator seats at `~2.857%` each + 20% across the 4 Independent Miner seats (`8.00%` for 1 Coin-Lock + `4.00%` each for 3 Open PoW); spillover seats earn the filling validator's own track rate), while unsolved seats receive `0%` and the Coordinator retains its full 15% reward when rescuing a stalled block with standby validators—splitting any unclaimed shares (`nUnclaimedPartSplit`) **50% to the Automatic Protocol Burn (`nExpectedBurn = nUnclaimedPartSplit / 2`)** and **50% to the Developer Treasury (`nExpectedTreasury = nTreasurySplit + (nUnclaimedPartSplit - nExpectedBurn)`)**.

If the quorum is met, the Coordinator signs the rolling seed to produce `vAdamVRFProof` and signs the final block header (`vAdamCoordinatorSig`) using the hybrid BLS12-381 + ECDSA fallback signature mechanism (`SignBLSWithECDSAFallback`).

6. Cooperative Proof-of-Stake (PoS)

At blocks ≥ 200, the consensus integrates with PoS. The staker's wallet validates the kernel hash difficulty (using the cumulative `CalculateMPAWeight()`). Once a valid staking UTXO is found, the staker's wallet retrieves the elected miners for the current block height via `SelectAdamNodes` and collects the lightweight PoW puzzles solved by these elected miners from the P2P network memory cache (`mapAdamSolutionsCache`). If the quorum threshold of solutions is not yet met, block template generation is deferred. Once the threshold is met, if the active Coordinator key is local to the node, it signs `vAdamVRFProof` and `vAdamCoordinatorSig` directly; if the active Coordinator resides on a remote Masternode or validator node, the staker broadcasts an ECDSA-authenticated `POSCOORDREQ` (`SignPoSCoordRequest`) over P2P, and the remote Coordinator verifies `VerifyPoSCoordRequestAuth`, runs `TestBlockValidity`, reserves its `AcquireCoordinatorProposalSlot`, and returns `POSCOORDSIG` (`vAdamVRFProof` + `vAdamCoordinatorSig`) before any LLMQ `vQuorumSig` request (`RequestBlockQuorumSig`). Finally, the staker signs the finished block header using the staking UTXO private key (`vchBlockSig`), locking the block under dual signatures (PoS Block Signature + ADAM Coordinator Signature) across fully independent nodes.

---

5. Security Analysis & Mitigations

The production implementation of ADAM in the KristaTech blockchain is hardened against several standard consensus vulnerabilities:

A. Sybil Attacks

- **Threat**: A malicious actor spins up thousands of cheap virtual private servers (Sybils) to gain a majority of voting power and control leader election.
- **Mitigation**: On Mainnet and Testnet, KristaTech restricts the election pool to verifiable on-chain Sybil-resistant tracks: active Masternodes (`4,200 KRISTA` collateral), Coin-Lock validators (`250 KRISTA` CLTV lock), and PoW-Lock / bootstrap validators (`10 KRISTA` CLTV + 28-bit PoW, with bounded `< 13` Zero-Coin 24-bit PoW and Blocks `1..199` coinbase fallback only during initial network bootstrap prior to `nAdamBootstrapLimit`). A deterministic 15-key cryptographic keypool fallback exists **exclusively on Regtest** for automated unit/functional test harnesses and is never active on Mainnet or Testnet. This makes Sybil attacks economically and computationally infeasible.

B. DDoS and Leader Targeting

- **Threat**: If the next block producer's IP address is known beforehand, attackers can DDoS the node to stall the blockchain.
- **Mitigation**: Leaders are elected using the rolling VRF seed. Nodes calculate their election rank locally. Because the seed is updated recursively and signatures are deterministic, the identity of the next leader remains hidden until they publish a signed block header, preventing preemptive targeting.

C. Sabotage, Network Latency & Deadlock Recovery

- **Threat**: Offline miners, network latency, or simultaneous failure of 5+ primary seats prevent the Coordinator from collecting enough solutions, halting block production.
- **Mitigation**: The network enforces a quorum threshold defined by `GetAdamThreshold(nHeight)` (`5` during bootstrap and `7` post-bootstrap on Mainnet/Regtest; `3` post-bootstrap on Testnet) out of `nAdamMinersCount = 11` elected miners, combined with two automatic recovery mechanisms: (1) **Same-Track Standby Seat Handover (**$t \geq 60$s**)** replacing silent primary seats with ranked standby validators from the same collateral track (`MN -> MN` , `Coin-Lock -> Coin-Lock` , `Open PoW -> Open PoW`), and (2) **Time-Relaxed Quorum Threshold (**$t \geq 120$s**)** relaxing the threshold by `1` every `60s` down to `ADAM_MIN_RELAXED_THRESHOLD = 5` . The **Orphan Solution Cache** holds out-of-order solutions, and the node-wide **`ActiveProposalGuard`** prevents dual PoW+PoS block proposals on the same parent block.

D. Dual Signature Security

- **Threat**: An attacker with 51% PoW or 51% PoS tries to reorganize the blockchain.
- **Mitigation**: Reorganizing the blockchain requires controlling *both* 51% of the staking weight and the private keys of the elected ADAM coordinator and miners. This dual-signature locking makes history reorganization virtually impossible.

---

6. Statistical Analysis

We analyze the mining hardware distribution to demonstrate the economic protection against ASICs.

Let:

- N be the total number of mining clients.

- K be the hash rate of a standard GPU client.
- FPGAs be $7\times$ more efficient than GPUs ($7K$).
- ASICs be $50\times$ more efficient than GPUs ($50K$).

Case 1: Competitive Race (Traditional PoW)

If 10% of the network consists of ASICs:

- Total ASIC power: $0.1N \times 50K = 5NK$
- Total GPU power: $0.9N \times K = 0.9NK$
- Ratio of ASIC blocks mined: $\frac{5}{5.9} \approx 84.7\%$

ASIC miners win almost 85% of all block rewards, centralizing the network and forcing GPU miners to leave.

Case 2: Cooperative Round (ADAM)

In ADAM, only the elected nodes participate. The probability P_e of an ASIC node being elected is proportional to its share of the active Masternode collateral, not its hashing power.

Once elected, the lightweight puzzle is solved instantly by both GPUs and ASICs (since the target is relaxed by $2^{\text{activeShift}}$).

- The participant validator pool (40% of the net block reward under Model D) is distributed deterministically across all $M = 11$ committee seats according to their track quota (`8.00%` for the 1 Coin-Lock seat, `4.00%` per seat across the 3 Open PoW seats and `~2.857%` per seat across the 7 Masternode Validator seats).
- The maximum reward a single elected seat can earn per block is strictly bounded to its fixed single-seat share, regardless of whether it hashes with a consumer GPU or an industrial ASIC.
- Investing in expensive hashing ASICs yields no additional per-seat block rewards, making ASIC development economically unviable.

References

- **[1]** Nakamoto, S. (2008). *Bitcoin: A Peer-to-Peer Electronic Cash System.*
- **[2]** Ravencoin Team. (2018). *X16R Whitepaper.*